SyncNature

Signature Video Analyzer

Numerical Analysis — Project Report



Saba Chichinadze

May 24, 2026

Implemented in Python (Flask, OpenCV, NumPy, SciPy) with a browser-based frontend.

Running locally at `http://localhost:5000`

Contents

1 Project Idea and Motivation **2**

2 Real-World Relevance **2**

3 System Description **3**

4 Implementation **4**

4.1 Technology Stack 4

4.2 Method 1 — Edge Detection 4

4.2.1 Implementation 4

4.2.2 Visualization 5

4.3 Method 2 — Interpolation 6

4.3.1 Implementation 6

4.3.2 Visualization 7

4.4 Method 3 — Derivatives 7

4.4.1 Implementation 7

4.4.2 Visualization 8

4.5 Method 4 — Rootfinding 8

4.5.1 Implementation 8

4.5.2 Visualization 10

5 Challenges and Lessons Learned **12**

6 Potential Improvements **13**

7 Summary **13**

1. Project Idea and Motivation

Signature analysis has been an area of personal interest for some time. In previous work I explored shape approximation of handwritten signatures, specifically fitting curves to their geometric form. What I had not yet examined was the dynamic component: how a signature is actually drawn, not just what it looks like when finished.

The question that motivated this project was straightforward. A completed signature is a static image. But the act of writing it carries temporal information (velocity, acceleration, hesitation points) that disappears the moment the pen leaves the paper. Could a numerical pipeline extract and quantify that information from video?

This direction aligned naturally with the course material. Signature dynamics require exactly the four methods covered this semester: edge detection to locate the pen in each frame, interpolation to reconstruct the continuous path between measurements, derivatives to extract velocity and acceleration from discrete position data, and rootfinding to find the precise moments where speed reaches a local minimum.

2. Real-World Relevance

Handwriting dynamics have practical applications in several fields.

In **forensic document examination**, analysts study not just the shape of a signature but its pressure, rhythm, and timing: characteristics that are difficult to forge. A forger can copy the visual form of a signature, but replicating its dynamic profile requires reproducing the exact rhythm of acceleration and deceleration, which is far harder to imitate.

In **medical research**, changes in handwriting dynamics serve as early indicators of neurological conditions. Parkinson's disease, for instance, introduces tremor and velocity irregularities in writing that appear before other symptoms become obvious. Quantitative analysis of pen dynamics can provide clinicians with an objective, reproducible measurement.

In **behavioral biometrics**, dynamic signature features form the basis of systems that authenticate identity not from what is written but from how it is written. The speed profile, the number and location of hesitation points, and the acceleration rhythm together constitute a behavioral fingerprint that is difficult to replicate.

The current project addresses the analytical foundation underlying all of these applications: extracting a reliable, mathematically grounded description of pen motion from raw video data.

3. System Description

SyncNature takes a video of a signature being drawn on a dark background with a bright-colored pen and processes it through a sequential pipeline of four numerical methods. The user either records a signature directly in the browser or uploads an existing video file. Python processes the video server-side and returns all results to the browser for visualization.

The pipeline is strictly sequential:

$$\text{Video} \longrightarrow \underbrace{\text{Edge Detection}}_{\text{Method 1}} \longrightarrow \underbrace{\text{Interpolation}}_{\text{Method 2}} \longrightarrow \underbrace{\text{Derivatives}}_{\text{Method 3}} \longrightarrow \underbrace{\text{Rootfinding}}_{\text{Method 4}} \quad (1)$$

Each method receives its input from the previous one. The output of each method is both used by the next method and visualized independently for the user.

The system runs locally. The backend is started with `python app.py` and is accessible at `http://localhost:5000`. The frontend is a single HTML file served by Flask.

4. Implementation

4.1. Technology Stack

Tool	Role
Python	Core computation environment for all four methods
Flask	Local web server; exposes the `/analyze` endpoint
OpenCV	Frame extraction, color thresholding, Gaussian blur, Sobel operator
NumPy	Array operations, `gradient` for central difference, `convolve` for smoothing
SciPy	`CubicSpline` for interpolation, `savgol_filter` for derivative smoothing
HTML5 Canvas	Browser-side signature drawing and video capture
MediaRecorder API	WebM video recording in the browser

Table 1: Tools used and their roles in the system.

4.2. Method 1 — Edge Detection

4.2.1. Implementation

The video is read frame by frame using OpenCV. For each frame, a color threshold isolates the cyan stroke pixels. Cyan on a black background has characteristically high blue and green channel values with low red:

$$\text{mask}[t] = (B > 80) \ \wedge \ (G > 60) \ \wedge \ (R < 120) \tag{2}$$

The discrete time derivative of the mask reveals which pixels are newly drawn in each frame:

$$\text{new_pixels}[t] = \text{mask}[t] \ \wedge \ \neg\,\text{mask}[t-1] \tag{3}$$

The centroid of these new pixels gives the pen tip position for that frame:

$$\text{tip}_x = \frac{1}{|\mathcal{N}|} \sum_{(x,y)\in\mathcal{N}} x, \qquad \text{tip}_y = \frac{1}{|\mathcal{N}|} \sum_{(x,y)\in\mathcal{N}} y \tag{4}$$

where $\mathcal{N}$ is the set of new pixels. In testing, this approach captured a substantially higher proportion of frames than skeleton-based tip tracking, because it fires whenever any new ink appears rather than requiring new skeleton pixels to appear specifically.

A Gaussian kernel $[1, 2, 4, 2, 1]/10$ is convolved with the coordinate sequences to remove frame-to-frame centroid jitter without distorting the overall trajectory shape.

For the geometric edge visualization, the Sobel operator is applied to the final complete frame. A Gaussian blur precedes the Sobel computation to suppress single-pixel noise:

$$G_x = \frac{\partial I}{\partial x}, \qquad G_y = \frac{\partial I}{\partial y}, \qquad G = \sqrt{G_x^2 + G_y^2} \tag{5}$$

Points exceeding 30% of the maximum G value are retained as edge points.

Listing 1: Core edge detection loop.

```
for i in range(1, len(frames)):
    curr_mask  = get_mask(frames[i])
    new_pixels = curr_mask & ~prev_mask
    if np.sum(new_pixels) >= 3:
        ys, xs = np.where(new_pixels)
        pen_positions.append({
            'x': float(np.mean(xs)),
            'y': float(np.mean(ys)),
            't': float(i) / fps * 1000.0
        })
    prev_mask = curr_mask

# Sobel on final frame
sobelx = cv2.Sobel(blurred, cv2.CV_64F, 1, 0, ksize=3)
sobely = cv2.Sobel(blurred, cv2.CV_64F, 0, 1, ksize=3)
sobel_mag = np.sqrt(sobelx**2 + sobely**2)
```

4.2.2. Visualization

The left panel shows the pen tip trajectory in cyan, tracing the path the pen traveled frame by frame. The right panel shows the Sobel edge points in red, forming a dot-outline

of the complete signature boundaries.

4.3. Method 2 — Interpolation

4.3.1. Implementation

The pen positions from Method 1 are discrete spatial samples at approximately 15 frames per second. Between consecutive frames the pen moved continuously, but that motion is unobserved. Cubic spline interpolation reconstructs a smooth continuous path through all detected positions.

Arc-length parameterization is used to avoid speed-dependent sampling bias. Rather than parameterizing by time, the parameter increases by Euclidean distance between consecutive positions:

$$s[0] = 0, \qquad s[i] = s[i-1] + \| \mathbf{pos}[i] - \mathbf{pos}[i-1] \| \tag{6}$$

This ensures that evaluating the spline at uniform values of s produces uniformly spaced points along the path regardless of the original drawing speed.

Two cubic splines are fitted independently, $x(s)$ and $y(s)$, using `scipy.interpolate.CubicSpline`. Between consecutive knots s_i and s_{i+1}:

$$x(s) = a_i + b_i(s - s_i) + c_i(s - s_i)^2 + d_i(s - s_i)^3 \tag{7}$$

The spline is C^2-continuous: position, first derivative, and second derivative are all continuous at every knot. It is an interpolant, not an approximation: the residual at each knot is exactly zero, which is verified and reported in the system log.

Listing 2: Arc-length parameterization and spline fitting.

```
diffs    = np.diff(pts, axis=0)
seg_lens = np.linalg.norm(diffs, axis=1)
arc_s    = np.concatenate([[0.0], np.cumsum(seg_lens)])

# Remove duplicate arc-length values (required by CubicSpline)
_, unique_idx = np.unique(arc_s, return_index=True)
arc_s_u = arc_s[unique_idx]
pts_u   = pts[unique_idx]

```

```
cs_x = CubicSpline(arc_s_u, pts_u[:, 0])
cs_y = CubicSpline(arc_s_u, pts_u[:, 1])

s_uniform = np.linspace(0.0, total_len, N)
curve = [{'x': float(cs_x(s)), 'y': float(cs_y(s))}
         for s in s_uniform]
```

4.3.2. Visualization

The left panel overlays the smooth green spline curve on the raw cyan dot positions, making the interpolation effect directly visible. The right panel shows the spline curve alone. The system log confirms the arc length and that the maximum knot residual is ≈ 0.

4.4. Method 3 — Derivatives

4.4.1. Implementation

The pen positions carry actual timestamps derived from the frame index and the computed frame rate. The spacing between consecutive positions is non-uniform, because whenever a frame produced no new pixels it was skipped, creating variable gaps. `numpy.gradient` handles this correctly by applying the central difference formula with the actual timestamp values:

$$v_x[i] = \frac{x[i+1] - x[i-1]}{t[i+1] - t[i-1]} \quad \text{(interior)}, \qquad v_x[0] = \frac{x[1] - x[0]}{t[1] - t[0]} \quad \text{(endpoint)} \tag{8}$$

Speed is the magnitude of the velocity vector:

$$\text{speed}[i] = \sqrt{v_x[i]^2 + v_y[i]^2} \tag{9}$$

Acceleration is computed by differentiating the velocity components:

$$a_x[i] = \frac{v_x[i+1] - v_x[i-1]}{t[i+1] - t[i-1]}, \qquad \text{accel}[i] = \sqrt{a_x[i]^2 + a_y[i]^2} \tag{10}$$

The signed derivative of speed, ds/dt, is computed from the speed array using the same formula. This quantity is negative while the pen decelerates and positive while it accelerates; its zeros mark the speed minima passed to Method 4.

Savitzky-Golay smoothing (window 15, polynomial degree 3) is applied to speed, accel-

eration, and ds/dt. This filter removes noise while preserving the height and position of peaks, which simple moving-average smoothing does not.

Listing 3: Central difference derivatives with non-uniform timestamps.

```
vx    = np.gradient(pts[:, 0], t_sec)
vy    = np.gradient(pts[:, 1], t_sec)
speed = np.sqrt(vx**2 + vy**2)

ax    = np.gradient(vx, t_sec)
ay    = np.gradient(vy, t_sec)
accel = np.sqrt(ax**2 + ay**2)

ds_dt = np.gradient(speed, t_sec)

speed_s = savgol_filter(speed, win, polyorder)
ds_dt_s = savgol_filter(ds_dt, win, polyorder)
accel_s = savgol_filter(accel, win, polyorder)
```

4.4.2. Visualization

The left panel shows the normalized speed profile in cyan. The right panel shows the normalized acceleration profile in orange. Both profiles are scaled to $[0, 1]$ for display while preserving the shape of all features.

4.5. Method 4 — Rootfinding

4.5.1. Implementation

The target function is ds/dt, the signed rate of speed change from Method 3. A speed minimum satisfies $ds/dt = 0$. These minima correspond to genuine pauses, hesitations, or direction reversals: the moments where the pen was at its slowest.

A `CubicSpline` is fitted to the ds/dt array, providing an analytic first derivative for Newton-Raphson.

The algorithm scans for sign changes where ds/dt transitions from negative to positive. Each such transition identifies a minimum. Three filter conditions are applied before bisection:

- Local speed must be below 20% of the maximum, confirming a genuine near-stop

- The dip must be at least 15% below surrounding values, eliminating noise-induced crossings
- Strict sign tolerance: $f[i] \cdot f[i+1] < -10^{-6}$, excluding flat plateaus

Phase A: Bisection. For each qualifying interval $[i,\ i+1]$, the interval is halved iteratively:

$$\text{mid} = \frac{a+b}{2}, \quad \begin{cases} b = \text{mid} & \text{if } f(a) \cdot f(\text{mid}) < 0 \\ a = \text{mid} & \text{otherwise} \end{cases} \tag{11}$$

The Intermediate Value Theorem guarantees convergence: if $f(a)$ and $f(b)$ have opposite signs and f is continuous, a root exists in $[a, b]$. The iteration terminates when $|b-a| < 10^{-6}$.

Phase B: Newton-Raphson refinement. Starting from the bisection result x:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \tag{12}$$

where $f(x_n)$ is the CubicSpline evaluated at x_n and $f'(x_n)$ is its analytic first derivative. Convergence is quadratic. The iteration terminates when $|x_{n+1} - x_n| < 10^{-6}$.

Listing 4: Bisection followed by Newton-Raphson refinement.

```
# Phase A: Bisection
a, b = float(i), float(i + 1)
for _ in range(MAX_ITER):
    mid = (a + b) / 2.0
    if cs_dsp(a) * cs_dsp(mid) < 0:
        b = mid
    else:
        a = mid
    if (b - a) < EPS:
        break
x = (a + b) / 2.0

# Phase B: Newton-Raphson
for _ in range(MAX_ITER):
    f  = float(cs_dsp(x))
    fp = float(cs_dsp(x, 1))
    if abs(fp) < 1e-12:
```

```
        break
    x_new = float(np.clip(x - f/fp, 0.0, float(n-1)))
    if abs(x_new - x) < EPS:
        x = x_new
        break
    x = x_new
```

4.5.2. Visualization

The left panel shows the speed profile in green with yellow vertical markers at each discovered minimum. The yellow dot sits at the exact bottom of the speed dip. The right panel shows the ds/dt profile in yellow, the function whose zeros were found, with a horizontal dashed line at zero confirming that the markers correspond to genuine sign changes.



Figure 1: Analysis results: edge detection, interpolation, velocity and acceleration profiles.

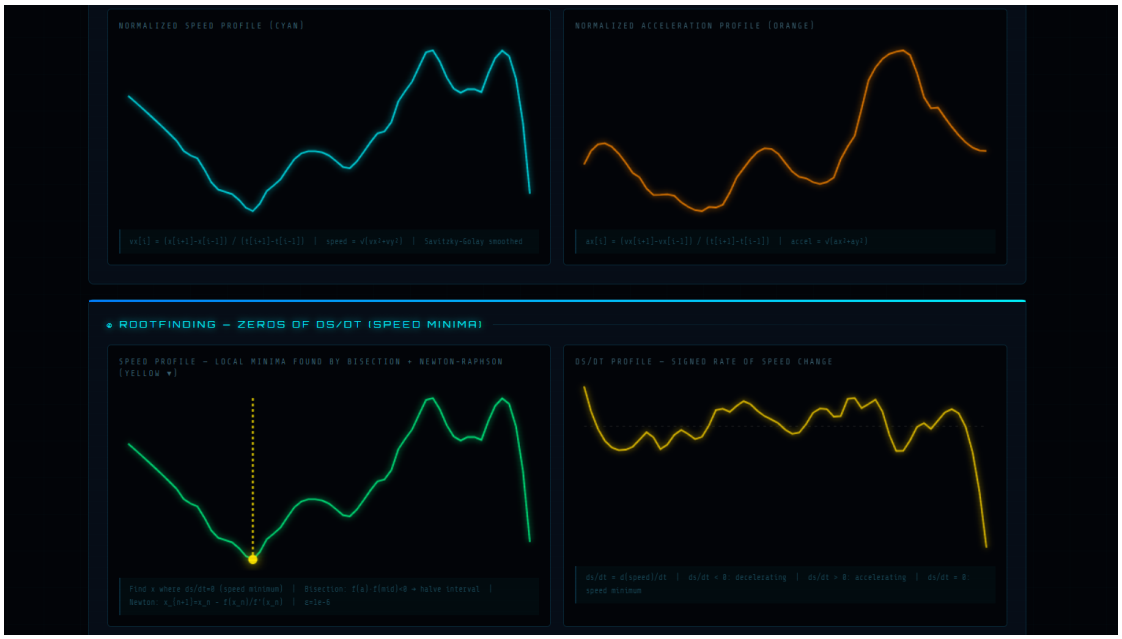


Figure 2: Rootfinding output: speed minima located by bisection and Newton-Raphson, and the ds/dt profile showing zero crossings.

5. Challenges and Lessons Learned

Rootfinding placement. The most significant conceptual challenge was determining where rootfinding genuinely belongs in the pipeline. An early approach attempted to find zeros of raw speed, which almost never reach exactly zero in a continuously written signature. The mathematically correct target is ds/dt, whose zeros are always present wherever the pen has a speed minimum. Understanding this distinction required careful reasoning about what rootfinding actually requires: a function with a sign change, not merely a function that is small.

Video versus canvas coordinate mismatch. An early design had one recording processed from video and another from browser canvas mouse coordinates for comparison. These two sources produce velocity profiles that are incomparable in scale and temporal resolution: video operates at 15 fps with pixel-level centroid positions, while canvas events fire at the browser event rate with subpixel coordinates. Any similarity score computed across these two sources proved unreliable. The lesson was that meaningful comparison requires the same instrument applied consistently to both inputs.

Pen lift detection and interpolation conflict. Detecting pen lifts through rootfinding requires recording stationary positions during lift gaps so that speed reaches zero. However, interpolation receiving those same positions draws a connecting line through the gap, making separate strokes appear as one continuous path. These two requirements are contradictory given a single shared position array. The resolution was to apply the four methods cleanly to continuous strokes, with pen lift detection deferred to a future preprocessing stage.

False pen lift detection. An early pen lift counter incremented whenever the centroid jumped more than a threshold distance between frames. Crossing over a previously drawn stroke caused the centroid to jump to the intersection region, registering as a lift when none had occurred. The correct approach requires a temporal condition (consecutive frames with no new pixels at all) rather than a spatial jump condition.

General lesson. Building this system clarified something easily overlooked: numerical methods are precise tools with specific mathematical requirements. Rootfinding requires a function with a sign change. Interpolation requires knowing what is being interpolated. Derivatives require consistent time spacing or explicit handling of non-uniform intervals. Applying these methods to real, noisy, video-derived data makes those requirements

concrete in a way that textbook examples alone cannot.

6. Potential Improvements

Identity verification. The original motivation for the project included using the extracted dynamics for signature verification, specifically comparing the speed profile and rootfinding results of two recordings of the same signature. The analytical foundation is in place. Verification would require establishing a similarity metric and a threshold that reliably separates genuine signatures from impostors.

Higher frame rate input. At 15 fps, derivative computation has limited temporal resolution: velocity is computed from position changes across 67 ms intervals. A 60 fps input would reduce this to 17 ms, producing a finer speed profile and more precise rootfinding results.

Pen lift handling. A preprocessing stage could detect lift gaps in the frame sequence before passing positions to the four-method pipeline. Each continuous stroke segment could then be processed independently, with lift counts maintained separately from the derivative analysis.

Alternative stroke media. The current system expects cyan strokes on a black background, a constraint imposed by the browser canvas. Processing videos of ink on paper would require a more general edge detection approach, such as adaptive thresholding or a learned stroke segmentation model.

7. Summary

SyncNature processes a recorded signature video through four sequential numerical methods (edge detection, cubic spline interpolation, derivative computation, and rootfinding) to extract a mathematical description of the dynamics of handwriting. The system is implemented in Python with Flask serving a browser-based recording and visualization interface. Each method produces a distinct visual output that makes the underlying mathematics directly observable.

The project demonstrated that standard numerical methods, applied carefully to real video data, can extract meaningful physical information, namely pen velocity, acceleration, and

speed minima, from something as ordinary as a handwritten signature. The gap between a clean textbook formulation and a working implementation on noisy, real-world data turned out to be precisely where the most useful learning occurred.